UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

State Management using Redux in Large React Applications

Case Study

Subject Name: Frontend Technologies (25CAH-655)

Submitted to:-
Ms. Deepali Saini Ma'am

Submitted by: -

Name: Nancy

UID: 25MCA20253

Section: 25MCA-2-A

In the partial fulfillment of requirements for the award of the Degree in

Master of Computer Application

Chandigarh University (CU)

Batch 2025-2027

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to everyone who has supported and guided me throughout the completion of this case study.

I am deeply thankful to Ms. Deepali Saini Ma'am for her valuable guidance, encouragement, and constant support. Her insightful suggestions have been instrumental in enhancing the quality of my work.

I would also like to extend my thanks to Chandigarh University and the Department of Master of Computer Applications (MCA) for providing an excellent academic environment.

My special thanks go to my classmates, friends, and family members for their encouragement and support.

Lastly, I wish to acknowledge the valuable assistance of. various online learning platforms, GitHub repositories, AI tools, and official documentation

ABSTRACT

Large React applications face many problems when managing shared data across multiple components. These problems include props drilling (passing data through many components), data inconsistency, and difficulty in scaling.

This case study explains how Redux solves these problems. Redux provides a central store that holds all application data. It uses actions, reducers, and dispatch to update data in a predictable way.

The study covers React basics like components and JSX. It also explains how to communicate with servers using Fetch API and REST APIs. Asynchronous operations are handled using Redux Thunk.

A real-world e-commerce example shows how to implement user authentication, product listing, shopping cart, and order placement. The case study also discusses advantages, limitations, testing, and deployment.

INTRODUCTION

1.1 Overview

This case study examines state management problems in large React applications and explains how Redux solves them. React is a popular library for building user interfaces. However, as applications grow, managing shared data becomes difficult.

Redux provides a central store, read-only state, and pure functions to modify state. This study provides clear explanations, code examples, and practical applications.

1.2 Background and Need for the System

Traditional React applications face several problems:

- **Props Drilling**: Data must be passed through many components that don't need it.
- **Data Inconsistency**: Different components maintain their own copies of the same data.
- **Poor Scalability**: As features are added, data flow becomes complex and hard to manage.
- **Debugging Difficulty**: Tracking when and why data changes is very hard.
- **Performance Issues**: Components re-render unnecessarily, slowing down the application.

Redux solves these problems by providing a single, central place for all application data.

1.3 Objectives of the Project

1. To understand the challenges of state management in large React applications.
2. To explain Redux concepts: store, actions, reducers, dispatch, and selectors.
3. To demonstrate unidirectional data flow and predictable state updates.
4. To explain React fundamentals: components, hooks, and JSX.
5. To implement server communication using Fetch API and REST APIs.
6. To handle asynchronous operations using Redux Thunk.
7. To discuss advantages, limitations, testing, and deployment.

PROBLEM STATEMENT

Problem Identification:-

1. Props drilling makes code hard to maintain.
2. Data inconsistency causes bugs.
3. Debugging state changes is very difficult.
4. Team development is slowed down.
5. Performance degrades due to unnecessary re-renders.

Proposed Solution:-

1. Use Redux as a central state management solution.
2. Maintain a single source of truth for all application data.
3. Use pure reducers to specify how state changes.
4. Use Redux Thunk for API calls and async operations.
5. Use Redux DevTools for easy debugging.

OBJECTIVE

1. **To use Redux** – To provide a central store that eliminates props drilling and data inconsistency in large React applications.
2. **To implement Redux Thunk** – To handle asynchronous API calls like fetching products and placing orders.
3. **To enable predictable state updates** – To ensure every state change happens through actions and reducers, making the code easy to debug.
4. **To improve application performance** – To use memoized selectors so components re-render only when needed.

5. **To eliminate props drilling** – To allow any component to access state directly from the Redux store without passing data through multiple components.
6. **To provide easy debugging** – To use Redux DevTools for time-travel debugging and inspecting every action.
7. **To build a shopping cart example** – To demonstrate adding items, removing items, and calculating totals using Redux state.
8. **To test Redux code** – To write unit tests for reducers using Jest to ensure state updates work correctly.
9. **To deploy a React-Redux application** – To create a production build and deploy it to platforms like Vercel.
10. **To understand when to use Redux** – To learn that Redux is best for large applications, while small apps can use useState or Context.

METHODOLOGY

Step 1: Requirement Analysis

- Found out what problems React apps face with state management.
- Decided to use Redux Toolkit, Redux Thunk, and Fetch API.

Step 2: Design

- Created a central store to hold all data.
- Divided state into slices: cart, products, and auth.

Step 3: Implementation

- Built React components using JSX.
- Created Redux slices with reducers for add, remove, and clear cart.
- Used useDispatch and useSelector to connect React with Redux.

- Used createAsyncThunk to fetch products from an API.

Step 4: Testing

- Wrote unit tests for reducers using Jest.

Step 5: Deployment

- Built the production version using npm run build.
- Deployed to Vercel.

ANALYSIS

.

1. **Problem Analysis:** Large React applications suffer from props drilling, data inconsistency, and debugging difficulty. There is no single source of truth.
2. **Requirement Analysis:** The solution needs a central store, predictable updates, async support, and good debugging tools.
3. **Functional Analysis:**
   - Store holds all state
   - Actions describe changes
   - Reducers specify how state changes
   - Dispatch sends actions
   - Selectors read data
4. **Performance Analysis:** Redux with selectors prevents unnecessary re-renders. Normalized state provides fast lookups.
5. **Feasibility Analysis:** Redux is free, well-documented, and widely used. It reduces development time for large applications.

CONCLUSION

wState Management using Redux successfully solves the problems of props drilling, data inconsistency, and debugging difficulty in large React applications.

Redux provides a central store, predictable updates, and excellent debugging tools. Redux Thunk handles API calls easily. The e-commerce example shows how to manage shopping cart, products, and orders.

This case study demonstrates how Redux helps build maintainable, scalable, and reliable React applications. For large applications, Redux is an excellent choice for state management.

LEARNING OUTCOMES

1. Learned how Redux solves props drilling by providing a central store for all application data.
2. Understood core Redux concepts: store, actions, reducers, dispatch, and selectors.
3. Implemented asynchronous API calls using Redux Thunk to fetch products from a backend server.
4. Connected React components to Redux store using useDispatch and useSelector hooks.
5. Built a functional shopping cart with add, remove, and checkout features using Redux state.

REFERENCES

1. GitHub Link - https://github.com/Nancy5848/Case-Study---State-Management-using-Redux-in-Large-React-Applications
2. Redux Documentation - https://redux.js.org/
3. React Documentation - https://react.dev/
4. Redux Toolkit Documentation - https://redux-toolkit.js.org/
5. React Redux Documentation - https://react-redux.js.org/